

LC847 - SHORTEST PATH VISITING ALL NODES

TCS

HCL

URBANCLAP

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 27, 2026

1. State Representation

we need to know which nodes have been visited till now so the state becomes:

(node, visited mask)

- **node** = current node
- **0001** → visited node 0
- **0101** → visited node 0, 2, etc.

so the Queue stores:

{node, {mask, distance}}

*apart from that we need **seen[node][mask]***

2. Multi-Source BFS Approach

since we can start from any node we will push every node initially
initially sabko as a source maanke push
mltisource bfs

- `{0, {0001, 0}}`
- `{1, {0010, 0}}`
- etc.

no need to use level wise bfs as every state knows its dist
we don't have anything to do with level

3. State Transition & Traversal

Suppose → `node = 2` , `mask = 0101`

neighbour = 3

then

```
newmask = mask | (1 << 3)
```

so push:

```
{3, {1101, d+1}}
```

4. Termination Condition

End traversal when:

```
mask == (1 << n) - 1
```

5. C++ Implementation

```
class Solution {
public:
    int shortestPathLength(vector<vector<int>>& graph) {

        int n=graph.size();

        //we need to maintain {node,{seenmask,dist}}
        //and a seen[node][seenmask]
        queue<pair<int,pair<int,int>>> q;
        vector<vector<int>> seen(n,vector<int>((1<<n),0)); //basically  $2^n$  states possible

        //initially sabko as a source maanke push
        //mltisource bfs

        for(int i=0;i<n;i++){

            int currmask=1<<i;
            q.push({i,{currmask,0}});
            seen[i][currmask]=1;
        }

        while(!q.empty()){
            //no need to use level wise bfs as every state knows its dist
            //we dont have anything to do with level

            int currnode=q.front().first;
            int currmask=q.front().second.first;
            int currdiss=q.front().second.second;

            q.pop();
            if(currmask==((1<<n)-1)) return currdiss;

            for(auto &newnode:graph[currnode]){

                int newmask=currmask | (1<<newnode);
                if(seen[newnode][newmask]) continue;

                seen[newnode][newmask]=1;
                q.push({newnode,{newmask,currdiss+1}});

            }
        }

        return -1;
    }
};
```

```
}  
};
```